

Enhancing Automated User Story Quality Assessment with Large Language Models

Guntur Budi Herwanto*

Department of Computer Science
and Electronics
Universitas Gadjah Mada
Yogyakarta, Indonesia
gunturbudi@ugm.ac.id

Wicaksono Leksono Muhamad

Department of Computer Science
and Electronics
Universitas Gadjah Mada
Yogyakarta, Indonesia
wicaksonoleksonomuhamad2001@ugm.ac.id

Waffiq Maaroja

Department of Computer Science
and Electronics
Universitas Gadjah Mada
Yogyakarta, Indonesia
waffiqmaaroja@ugm.ac.id

Abstract—User stories are a cornerstone of agile requirements engineering, and their quality is critical to project success. While frameworks like the Quality User Story (QUS) and tools like the Automatic Quality User Story Artisan (AQUSA) have advanced automated quality assessment, they remain limited to rigid templates and syntactic analysis, struggling with semantic and pragmatic criteria. This paper investigates the use of Large Language Models (LLMs) to overcome these limitations. We propose an augmented assessment pipeline where LLMs serve as a pre-processor to accurately extract components from non-standard user story templates and as an analyzer for complex semantic and pragmatic quality dimensions. Evaluated on a curated dataset of varied templates and the established Trident benchmark, our LLM-based approach significantly outperforms the rule-based AQUSA baseline in extraction accuracy and defect detection for criteria like Atomicity and Minimalism. The results demonstrate that LLMs offer a robust and generalizable path to more holistic automated user story quality assessment, reducing false positives and enabling teams to focus on higher-value tasks.

Index Terms—User Stories, Quality User Story Framework, LLM

I. INTRODUCTION

User stories are fundamental artifacts in agile software development, capturing functional requirements concisely from an end-user perspective [1]. Their quality is critical: high-quality stories improve team understanding, enable more accurate effort estimation, and reduce implementation defects [2], [3].

To standardize and evaluate user story quality, researchers have developed structured assessment frameworks. The Quality User Story (QUS) framework outlines comprehensive criteria across syntactic, semantic, and pragmatic dimensions [1]. This framework was operationalized through the Automatic Quality User Story Artisan (AQUSA) tool, which uses natural language processing (NLP) to automatically detect quality defects [4].

Although AQUSA represents significant progress in automation, its effectiveness is largely limited to syntactic aspects of the QUS framework [4], [5]. While it successfully verifies structural correctness, template adherence, and basic

grammar, it struggles with semantic qualities such as clarity and meaning, as well as pragmatic concerns like feasibility, dimensions that require contextual and domain knowledge beyond the capabilities of rule-based NLP.

AQUSA's reliance on rigid templates also limits its practical applicability. Our preliminary analysis of 500 user stories from three companies reveals that only 23

The emergence of powerful Large Language Models (LLMs) offers a promising pathway to overcome these limitations [6] [7]. LLMs excel in contextual understanding, reasoning, and nuanced text interpretation, making them well-suited to handle diverse user story templates and assess semantic and pragmatic criteria that have previously resisted automation.

Although recent studies have explored LLMs in requirements engineering, most focus on generation rather than quality assessment. Moreover, existing approaches lack systematic evaluation across multiple quality dimensions and do not adequately address template variability.

This research investigates how LLMs can augment AQUSA to form a more comprehensive quality assessment pipeline. We position LLMs as complementary tools, enhancing the evaluation of syntactic, semantic, and pragmatic dimensions. Our contributions are:

- 1) An assessment pipelines via an LLM-based pre-processing step that handles flexible user story formats, mitigating AQUSA's template dependency.
- 2) A methodology for applying LLMs to assess user story quality criteria, complementing AQUSA's syntactic analysis.

Our findings suggest that LLMs can effectively identify complex quality defects beyond structural issues. This work supports the development of automated tools that offer practical benefits for agile teams striving to improve requirements quality.

II. BACKGROUND AND RELATED WORK

A. The Quality User Story Framework and AQUSA

The Quality User Story (QUS) framework [1] defines 14 criteria across three dimensions: syntactic, semantic, and pragmatic quality. AQUSA (Automatic Quality User Story Artisan)

*Corresponding author: gunturbudi@ugm.ac.id

implements these criteria using automated analysis [4]. it provides strong support for syntactic and some pragmatic aspects (e.g., sentence structure, uniformity) using Linguistic Parser, User Story Base, Analyzer, Enhancer, and Report Generator. Linguistic Parser tries to detect and extract the role, means, and ends by searching for the provided format. Overall, this approach uses a heuristic approach to extract user story parts; such an approach is limited to a rigid template. The quality of a user story is often undermined by dirty information, which includes inconsistencies, conflicting conventions, spelling mistakes, missing or obsolete entries, duplications, and other erroneous values. Handling these problems usually requires extensive preprocessing, but such work can become unnecessarily verbose and counterproductive. These challenges demonstrate that forecasting the quality of user stories for monitoring purposes depends strongly on the early identification and correction of these issues [5].

B. Large Language Models for Requirements Engineering

Recent research shows that LLMs can enhance requirements engineering by offering contextual understanding, reasoning, and domain adaptation [8]–[10]. Prompt engineering, or prompting, is the methodology of carefully composing instructions in natural language for domain-specific tasks. Rather than relying on programming languages, prompts allow the model to capture the user's intent directly in human language, enabling the automation of non-clerical activities, such as quality checking or violation detection of user stories. This approach helps bridge the gap between technical requirements and natural communication, allowing LLMs to interpret and act on tasks that traditionally required manual effort [11].

In agile projects, requirements are often written as user stories, which must balance clarity, simplicity, and testability. Recent work has explored using LLMs not only as passive assistants but as active agents that interact and collaborate in role-based dialogues to refine such stories. For example, the ALAS framework [12] implements two coordinated LLM agents, a product owner and a requirements engineer/ Which both iteratively improve the quality of user stories against established criteria (e.g., the INVEST principles). This demonstrates that LLM-based systems can go beyond isolated text classification or information extraction, offering structured collaboration to directly enhance requirement artifacts in practice.

LLMs have also been studied for their ability to generate and evaluate user stories at scale [13]. A multi-agent setup was used in which analyst agents queried a customer agent to create almost fourteen thousand user stories. The quality of these stories was measured with syntactic checks (AQUSA) and semantic dimensions from the QUS framework, together with coverage, diversity, and detectability metrics. The results show that LLMs can reach human-level performance in coverage and structural clarity but produce less diverse stories and often fall short on deeper semantic aspects such as rationale or acceptance conditions. When detailed evaluation guidelines were provided, stronger models were able to score semantic qualities consistently with human raters, indicating

their potential as scalable support tools in agile requirements engineering.

III. METHODOLOGY

A. Research Questions

To evaluate the capabilities of Large Language Models (LLMs) in automating user story quality assessment, we address three research questions:

- 1) **RQ1:** How effectively can LLMs extract user story components from non-standard templates?
- 2) **RQ2:** How does the performance of an LLM-based assessment pipeline compare to the rule-based AQUSA tool?

RQ1 investigates whether LLMs can overcome the template rigidity of existing tools by accurately extracting user story components from varied formats. RQ2 compares the LLM-based pipeline against AQUSA, testing the hypothesis that LLMs excel at semantic and pragmatic assessments where rule-based methods fall short.

B. LLM Models and Implementation

We evaluated four LLM models:

- *Llama 4 Maverick*: A state-of-the-art 128-expert Mixture-of-Experts model for multilingual text understanding and enterprise applications [14].
- *Mistral (7B) Instruct*: A 7-billion-parameter model fine-tuned for instruction following and efficiency [15].
- *DeepSeek V3*: A Mixture-of-Experts model with 671B total parameters (37B activated per token) [16].
- *GPT-4.1 Mini*: A compact successor to GPT-4o with improved coding and instruction-following capabilities [17].

Models were accessed via heterogeneous APIs: Llama 4 Maverick and Mistral-7B through Together AI, and DeepSeek V3 and GPT-4.1 Mini via their native services. We standardized API calls using the LangChain framework to ensure consistent prompt routing and response processing across models [18].

C. Prompt Configuration

Our prompt engineering strategy employed a structured four-component approach to ensure consistent and reliable model responses:

- **OBJECT-CONTEXT**: Defines the structure of a user story (Role, Means, optional Ends).
- **INSTRUCTION**: Specifies the task ($\langle TASK \rangle$), definitions ($\langle DEFINITION \rangle$), and rules ($\langle RULES \rangle$).
- **INPUT DATA**: Provides the user story or components to evaluate ($\langle DATA \rangle$).
- **FORMATTING**: Requires output as a valid JSON object conforming to a strict schema ($\langle SCHEMA \rangle$).

Table I details the comprehensive prompt structure employed across all assessment tasks. The integration of chain-of-thought reasoning in the $\langle RULES \rangle$ component significantly enhanced the transparency and reliability of the assessment

TABLE I
GENERAL PROMPT CONFIGURATION

Variable	Prompt
OBJECT-CONTEXT	Based on the Quality User Story framework, a user story contains: [Role] + [Means] (+ optional [Ends])
INSTRUCTION	$\langle TASK \rangle$ $\langle DEFINITION \rangle$ $\langle RULES \rangle$
INPUT DATA	Given this data: $\langle DATA \rangle$
FORMATTING	Your response must be a single, valid JSON object. Do not include any text or formatting outside of the JSON structure. The JSON object should strictly conform to the following schema: $\langle SCHEMA \rangle$

process by requiring models to articulate their reasoning steps before reaching final conclusions.

D. Assessment Pipeline

To evaluate user story, we adapt the pipeline based on the AQUASA framework using LLM with three consecutive task: *Chunker* (extracting component or user story parts), *Template Extractor* (extracting user story template using extracted parts), and then *Analyzer* (analyzing the user story for each criteria using extracted parts), as shown in the Fig. 1. Each QUS criterion is evaluated in two ways: individually (without needing global context) or as part of a set (where global context matters) [1].

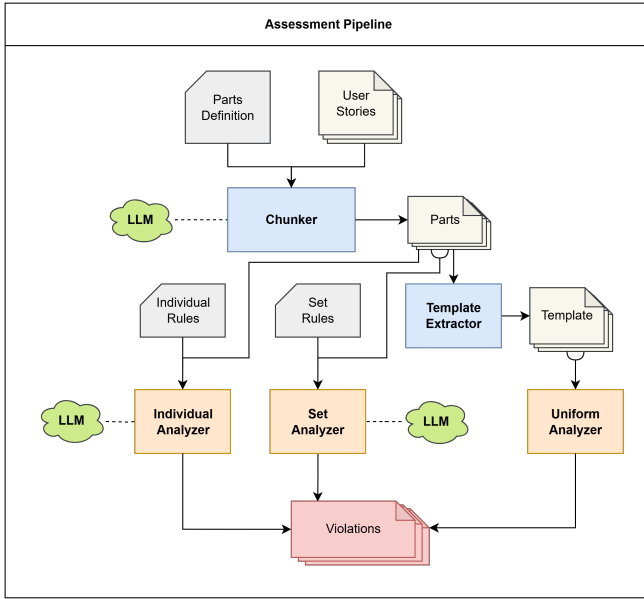


Fig. 1. Our proposed assessment pipeline

Chunker Component: The initial processing stage responsible for segmenting raw user story text into structured components (Role, Means, Ends). We implemented a preprocessing step that expanded all contractions to their full forms to ensure lexical consistency without altering semantic meaning. Three description granularity levels were tested: minimal (E), concise (S), and comprehensive (F), with each level injected into the $\langle DEFINITION \rangle$ portion of the prompt. The output schema included `role`, `means`, `ends`, and `expanded_text` fields,

providing a standardized representation for downstream processing.

Template Extractor Component: This intermediate stage addressed the critical challenge of template variation by standardizing diverse user story formats into a canonical template structure. Using the Stanza NLP library, the component performed tokenization and part-of-speech tagging to identify linguistic patterns. A custom Longest Common Subsequence (LCS) algorithm matched components identified by the Chunker against the tokenized text, replacing them with standardized tokens (ROLE, MEANS, ENDS). The output included the templated text, component POS patterns, appearance order, and any non-templated remnants, creating a normalized representation essential for consistent quality assessment.

Analyzer Component (Individual Criteria): This stage conducted comprehensive quality assessment across the QUS framework dimensions. Syntactic criteria (Atomic, Minimal, Well-formed) were handled through efficient rule-based checks, while semantic (Conceptually Sound, Problem-Oriented, Unambiguous) and pragmatic (Full Sentence, Estimable) criteria employed LLM-based analysis with detailed rule sets. The output schema included a valid boolean indicating overall compliance and a structured violations array detailing specific issues with component references, violation explanations, and remediation suggestions.

Analyzer Component (Set Criteria): For assessing quality dimensions requiring cross-story analysis, we implemented a pairwise comparison approach. The Uniform criterion utilized template-based comparisons, while Unique (duplicate detection), Independent (dependency identification), Complete (coverage verification), and Conflict-free (consistency checking) criteria employed LLM-based analysis of story pairs. For a collection of N user stories, the system performed $\frac{N(N-1)}{2}$ comparisons, ensuring comprehensive coverage of all possible relationships and enabling detection of complex quality issues that transcend individual story assessment.

IV. EXPERIMENTAL SETUP

A. Dataset

To evaluate the entire task in the pipeline, we utilized the separate dataset for Chunker and Analyzer. For the chunker, we create a new dataset because existing user story dataset is limited to standard template, thus can't be used to evaluate robustness of the Chunker method. To leverage the flexible template, first 100 user stories containing all parts was created using standard template with this composition:

- 25 user stories with single [Role] and using action verb for the [Means]
- 25 user stories with multiple [Role] and using action verb for the [Means]
- 25 user stories using passive verb for the [Means]
- 25 user stories using [Ends] as a qualitative effect of the [Means]

Each of them then augmented to produce a single user story with different template randomly selected from this 6 templates:

- In order to [Ends], a/an [Role] needs to [Means]
- I would like to [Means] as a [Role], because [Ends]
- From the perspective of [Role], it's necessary to [Means] because [Ends]
- For the [Role], [Means] must be possible because [Ends]
- To [Ends], a [Role] must [Means]
- As a/an [Role], [Ends] by [Means]

Sometimes user story can miss one of its parts. Thus from each of the original user story, a single user story with one of the part being removed is produced. The removed part is randomly selected between [Role], [Means], or [Ends]. Then we get a total of 300 user stories.

Meanwhile, for the Analyzer task, we used a real scenario set of user stories created by a Duke University team for the Trident project [1]. This dataset consists of five labeled criteria that were previously annotated by [19]. Each labeled set contains 48 stories with standard template, evaluated based on three syntactic criteria (Atomic, Minimal, Well-formed) and two pragmatic criteria (Uniform and Uniqueness).

B. Similarity Measurement

The main purpose of the Chunker method is to extract the substring from the user story related to each part, without adding or changing the words from that, which can alter the meaning of the extracted part or user story itself. Thus, the string matching algorithm is used to calculate the similarity between two strings, which can be used for evaluating extracted parts. Given two lists X and Y . We calculate the similarity score as:

$$Sim(X, Y) = \begin{cases} 1 & \text{if } |X| = |Y| = 0 \\ \frac{LCS(X, Y)}{\max(|X|, |Y|)} & \text{otherwise} \end{cases} \quad (1)$$

where $LCS(X, Y)$ is the length of the Longest Common Subsequence between list X and Y . This calculation applied to [Means] and [Ends], meanwhile for the [Role] the calculation is slightly different because the user story can have multiple [Role]. Given two lists X and Y , both contain the list of cleaned and tokenized words for each [Role] extracted from two user stories. We calculate the similarity score for [Role] as:

$$Sim_R(X, Y) = \max_{\sigma \in \pi(m)} \sum_{i=1}^m Sim(X_i, Y_{\sigma(i)}) \quad (2)$$

where $m = \max(|X|, |Y|)$ and $\pi(m)$ is the set of all permutations of $\{1, 2, \dots, m\}$. When $i > |X|$, then X_i is assumed to be an empty list, and the same goes for Y . This equation is

equivalent to calculating the maximal sum of similarity score between each pair across all bijective mappings from X to Y .

C. Evaluation Process

Overall evaluation separated based on the specific task and method used.

Chunker Assessment. For each user story in the first dataset, the parts are extracted using Chunker model and evaluated using the ground truth parts. Before evaluation, each parts needs to be cleaned by removing the non-alphanumeric and non-space character, then tokenized by splitting into the list of words. Given the M_0, E_0, M_1, E_1 as a list of words from extracted [Means] part, extracted [Ends] part, ground truth [Means] part, ground truth [Ends] part, respectively, and R_0 and R_1 as a list of list of words from each extracted [Role] part and each ground truth [Role] part, respectively. The overall similarity score between extracted parts and ground truth parts can be obtained by averaging the similarity score of [Role], [Means], and [Ends] part as follows.

$$\frac{Sim_R(R_0, R_1) + Sim(M_0, M_1) + Sim(E_0, E_1)}{3} \quad (3)$$

It evaluates using different prompt scenarios and LLM models explained before.

Uniformity Criteria Assessment. The template from each user stories on the second dataset is extracted using Template Extractor. The most frequent template is selected to be a general template, while removing some unnecessary terms from the template like 'a', 'an', and 'the'. For each user stories, it only violate the uniform criteria if its template is not identical to the general template, except for the [Ends] separator because [Ends] is optional. But if both [Ends] exist, then the separator must be identical. It evaluates using Chunker result from each LLM models.

Individual Criteria Assessment. Individual analyzer assessed the story for several individual criteria analyzed in AQUASA, e.g. *Atomic* and *Minimal* using best chunker model to extract the parts. It gives the violation list and evaluated using each LLM models. Meanwhile for the *Well-form* criteria, we use different scenario of chunker model instead of the best one because it not used LLM for analyzer.

Set Criteria Assessment. Set analyzer assessed the story only for set criteria analyzed in AQUASA, e.g. *Unique*, except for *Uniform*. It gives the violation list and evaluated using each LLM models.

D. Evaluation Metric

To quantitatively assess the performance of our LLM-based pipeline, we employ a suite of evaluation metrics tailored to each consecutive task. For the Chunker model, performance is measured through overall score of extracted parts, which gauges the correctness of the identified components against a ground truth. For the final Analyzer model, we employ standard classification metrics including False Positives (FP) and False Negatives (FN) to quantify its precision in detecting quality defects. Additionally, we track token usage to evaluate

TABLE II
CRITERIA ASSESSMENT RESULT (PER-MODEL)

Model	Atomic					Minimal					Well-formed					Uniform					Unique				
	DEF	FP	FN	P	R	DEF	FP	FN	P	R	DEF	FP	FN	P	R	DEF	FP	FN	P	R	DEF	FP	FN	P	R
GroundTruth	11	-	-	-	-	1	-	-	-	-	0	-	-	-	1	18	-	-	-	-	0	-	-	-	-
AQUSA	10	1	2	0.9	0.81	4	3	0	0.25	1.00	0	0	0	1	1	18	0	0	1.00	1.00	0	0	0	1	1
Llama	10	2	1	0.8	0.89	1	1	0	0	1.00	0	0	0	1	1	18	0	0	1.00	1.00	0	0	0	1	1
GPT	11	0	0	1.00	1.00	1	0	0	1.00	1.00	0	0	0	1	1	21	3	0	0.83	1.00	0	0	0	1	1
DeepSeek	13	2	0	0.85	1.00	1	0	0	1.00	1.00	0	0	0	1	1	20	2	0	0.90	1.00	0	0	0	1	1
Mistral	19	8	0	0.58	1.00	0	0	1	1.00	0	0	0	0	1	1	36	19	1	0.47	0.94	0	0	0	1	1

the model's efficiency and operational cost for this extraction task and quality assessment process.

TABLE III
COMPARISON OF CHUNKER PERFORMANCE ACROSS DIFFERENT MODEL AND PROMPTS

Prompt	Model	Score	#In	#Out
-	AQUSA	0.558	-	-
E	DeepSeek	0.816	313.71	66.86
	GPT	0.758	313.32	73.83
	Llama	0.812	316.16	75.46
	Mistral	0.728	360.41	85.64
S	DeepSeek	0.712	364.71	69.16
	GPT	0.791	362.32	73.62
	Llama	0.739	365.16	76.89
	Mistral	0.769	416.41	87.11
F	DeepSeek	0.832	601.71	71.89
	GPT	0.845	589.32	72.02
	Llama	0.849	592.16	74.31
	Mistral	0.764	663.41	87.68

V. RESULTS AND DISCUSSION

A. Chunker Assessment Result

The results in Table III show that all LLMs significantly outperformed the rule-based AQUSA baseline in extracting user story components. Performance improved with increased prompt detail, with the full-description prompt (F) yielding the highest accuracy. Llama achieved the best overall score of 0.849 using this prompt, followed closely by GPT and DeepSeek. However, this gain in precision came at a substantial cost, as evidenced by the sharp increase in input token consumption for prompt F. While Mistral was the most economical model in terms of token usage, it also delivered the lowest accuracy under the full-prompt condition. This establishes a clear trade-off between extraction accuracy and computational efficiency, with Llama offering the most favorable balance for this specific task.

TABLE IV
CHUNKER PERFORMANCE ON AUGMENTED TEMPLATES

Template	Average Score
1	0.8000
2	0.6509
3	0.8665
4	0.7346
5	0.6396
6	0.6330

Furthermore, the chunker's performance was evaluated on a dataset augmented with six distinct user story templates as shown in the Table IV. The results show significant variance in accuracy across templates, with scores ranging from 0.63 to 0.87. Template 3 yielded the highest average score (0.867), indicating the model excels with that specific syntactic structure. Conversely, Templates 5 and 6 proved most challenging, with scores near 0.63. This variance highlights that while the LLM-based chunker is highly effective, its performance remains sensitive to the structural phrasing of the input user story.

TABLE V
CHUNKER PERFORMANCE ON USER STORIES WITH MISSING COMPONENTS

Missing Component	Average Score
Role	0.86443
Means	0.33139
Ends	0.96722

The evaluation on user stories with missing components revealed a critical dependency on the means part as shown in the Table V. The model maintained high accuracy when the [Role] (0.864) or [Ends] (0.967) were absent, suggesting it can often infer or work around their omission. However, performance dropped drastically to 0.331 when the [Means] component was missing. This condition happens because the existence of verb in the [Ends] part makes this part mistakenly detected as [Means]. This indicates that the [Means] is the most critical and non-negotiable element for the chunker to reliably identify the structure of a user story, acting as its primary anchor.

B. Criteria Assessment Result

To evaluate the effectiveness of our approach, we compare the performance of a traditional heuristic baseline (AQUSA) against various LLM-based configurations, as detailed in Table II. These configurations include pairing specialized model chunkers with model evaluators. For the *Well-formed* and *Uniform* criterion, we vary the LLM model on the chunker. For the other criteria, we utilize Llama as the best chunker model and vary the LLM model on the analyzer. The evaluation employs the following key metrics:

DEF Total number of defects detected by the model (LLM or AQUSA).

FP Total number of false positives detected by the model.

TP Total number of true positives detected by the model.

Overall, the LLM-based approach improves precision while maintaining high recall when contrasted with the AQUASA baseline. On *Atomic*, GPT attains perfect precision and recall ($P = 1.00$, $R = 1.00$), improving over AQUASA and over models that introduce extra false positives. For *Minimal*, GPT and DeepSeek are consistently strong, with near-ceiling precision and recall, while AQUASA shows lower precision due to additional false positives. Under the model-as-chunker setting for *Uniform* and *Unique*, LLAMA and GPT remain robust, DeepSeek trails due to extra false positives, and Mistral underperforms because chunk boundary drift propagates to evaluation. For *Well-formed*, the ground truth contains no recorded violations, so results are trivially perfect across models and do not test violation detection.

VI. CONCLUSION

This work presents an LLM-based approach for structuring user stories into *[Role, Means, Ends]* and assessing five established quality criteria. Using the Trident dataset, we adopt a mixed evaluation strategy that selects LLAMA for *Uniform* due to its robustness and relies on multiple LLMs for the other criteria. The proposed method improves defect detection relative to AQUASA while reducing spurious flags, as shown in Table II. The result suggests that LLMs, paired with a reliable chunking step, can raise the baseline for automated user story review.

Our study is scoped to one dataset and five criteria, with incomplete model coverage for some cells in Table II. Results may be sensitive to prompts and inference settings, and we did not report human agreement alongside model judgments. We plan to extend the evaluation to additional domains and datasets, report precision, recall, and F1 consistently across criteria, study prompt and temperature stability, and explore model ensembling and rationale extraction. Integrating the approach into analyst workflows and measuring cost and latency will support practical adoption.

In sum, LLM-assisted analysis is a viable path to faster, more consistent user story quality checks, leaving human reviewers to focus on higher-value clarification and negotiation.

ACKNOWLEDGMENT

This research is supported by the Department of Computer Science and Electronics, Faculty of Mathematics and Natural Science, Universitas Gadjah Mada, Indonesia.

REFERENCES

- [1] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, "Improving agile requirements: the quality user story framework and tool," *Requirements Engineering*, vol. 21, no. 3, pp. 383–403, 2016.
- [2] G. Lucassen, F. Dalpiaz, J. Martijn E. M. Van der Werf, and S. Brinkkemper, "The use and effectiveness of user stories in practice," in *Requirements Engineering: Foundation for Software Quality*. Springer, 2016, pp. 205–222.
- [3] F. Dalpiaz and S. Brinkkemper, "Agile requirements engineering with user stories," in *IEEE International Conference on Requirements Engineering*. IEEE, 2018, pp. 336–337.
- [4] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, "Aqua: The automatic quality user story artisan for agile software development," *Information and Software Technology*, vol. 77, pp. 18–31, 2016.
- [5] A. Mustaffa, S. K. Rahimi, and R. Ahmad, "Enhancing quality of user stories using natural language processing: A systematic literature review," *Information and Software Technology*, vol. 138, p. 106631, 2021.
- [6] A. Ferrari and P. Spoletini, "Formal requirements engineering and large language models: A two-way roadmap," *Information and Software Technology*, vol. 181, p. 107697, 2025.
- [7] S. Minaee, T. Mikolov, N. Nikzad, M. A. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, "Large language models: A survey," *ArXiv*, vol. abs/2402.06196, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:267617032>
- [8] Y. Hou, X. Feng, and Y. Li, "Large language models for software engineering tasks: A comprehensive survey," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–35, 2023.
- [9] Y. Zhang, Y. Wu, and J. Li, "LLM-based user story assistant: A case study with agile teams," *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 189–203, 2024.
- [10] A. Sami, S. Nagarajan, and G. Kumar, "AI-assisted requirements engineering: Comparative analysis of LLMs for user story generation," *Journal of Systems and Software*, vol. 208, p. 111882, 2024.
- [11] K. Ronanki, B. Cabrero-Daniel, and C. Berger, *ChatGPT as a tool for User Story Quality Evaluation: Trustworthy out of the box?*, 12 2023. [Online]. Available: https://doi.org/10.1007/978-3-031-48550-3_17
- [12] Z. Zhang, M. Rayhan, T. Herda, M. Goisau, and P. Abrahamsson, *LLM-Based Agents for Automating the enhancement of user story quality: An early report*, 1 2024. [Online]. Available: https://doi.org/10.1007/978-3-031-61154-4_8
- [13] G. Quattrocchi, L. Pasquale, P. Spoletini, and L. Baresi, "Can LLMs generate user stories and assess their quality?" 2025. [Online]. Available: <https://arxiv.org/abs/2507.15157>
- [14] Meta, "The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation," 4 2025. [Online]. Available: <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>
- [15] A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. De Las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed, "Mistral 7B," *arXiv (Cornell University)*, 1 2023. [Online]. Available: <https://arxiv.org/abs/2310.06825>
- [16] DeepSeek-AI, A. Liu, B. Feng, and e. a. Xue, "DeepSeek-V3 Technical Report," *arXiv (Cornell University)*, 12 2024. [Online]. Available: <http://arxiv.org/abs/2412.19437>
- [17] J. Weatherbed and E. Roth, "OpenAI debuts its GPT-4.1 flagship AI model," 4 2025. [Online]. Available: <https://www.theverge.com/news/647896/openai-chatgpt-gpt-4-1-mini-nano-launch-availability>
- [18] O. Topsakal and T. C. Akinci, "Creating large language model applications Utilizing LangChain: A primer on developing LLM apps fast," *International Conference on Applied Engineering and Natural Sciences*, vol. 1, no. 1, pp. 1050–1056, 7 2023. [Online]. Available: <https://doi.org/10.59287/icaens.1127>
- [19] G. Lucassen, F. Dalpiaz, J. M. E. M. van der Werf, and S. Brinkkemper, "Improving user story practice with the grimm method: A multiple case study in the software industry," in *International Conference on Research Challenges in Information Science*. Springer, 2017, pp. 205–222.